ENGINEER INTEGRATION GUIDE

LineFollowerPipeline

Pure-Java, zero-dependency line-following vision processor

Single-file library • Java 8+ • No external dependencies • Updated October 2026 (v2)

What's new in this version (v2)

- **Distractor rejection.** A dark chair leg, a bag, a shadow or the dark floor past a table edge is no longer taken for the line: a band only counts as the line if its width and line-ness match the line measured at calibration (`rejectDistractors`, `bandMaxWidthMult`, `bandMinWidthMult`).
- **Steering uses the whole ⊥.** Steering = offset of the line in the scan strip (as before) + heading of the line from the look-ahead (new). A robot at an angle to the line now gets a steering value (±15° → ≈ ±8, ±30° → ≈ ±17). Range `-50 … +50` and sign are unchanged.
- **Hidden line is held.** A line that disappears mid-image is held for up to 6 frames (`trackHoldFrames`) instead of kicking steering to ±50 at once.
- **Steadier lock.** The gate follows the predicted line position and can no longer grow to the whole frame; a valid-looking lock is not stolen by a nearby object.
- **Richer callback.** `FrameData` adds `steerOffsetTerm`, `steerHeadingTerm`, `headingDeg`, `headingConfidence`, `lineHeld`, `rejectedBands`.
- **No API break.** Nothing was removed or renamed — existing integration code compiles and runs unchanged.

Changelog — What an Integrator Needs to Know NEW

In this guide NEW marks what v2 adds and CHANGED marks existing items whose behaviour changed in v2.

v2 — October 2026

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Distractor rejection — new `rejectDistractors`, `bandMaxWidthMult`, `bandMinWidthMult` (on by default)	A dark object next to the track (a black chair leg, a bag, a shadow, the dark floor past a table edge) is no longer taken for the line. A band only counts as the line if its **width** and **line-ness** match the line measured at calibration. See *Distractor Rejection*.	No
Steering uses the whole ⊥ — new `steerOffsetWeight`, `steerHeadingWeight`, `steerLookAhead`, `steerHeadingMinRows`, `steerHeadingCalibMaxDeg`	Steering = **offset** of the line in the scan strip (as before) **+ heading** of the line from the look-ahead (new). A robot at an angle to the line now gets a steering value even when the line crosses the strip at the calibrated column (±15° → about ±8, ±30° → about ±17). Range and sign are unchanged. See *Steering from the Whole ⊥*.	No — but the steering values differ when the robot is at an angle, so re-check your steering gain
Hidden line is held — new `trackHoldFrames`	If the line disappears in the **middle** of the image (glare, a shadow, an object over it), the last steering and tracking are held for up to 6 frames instead of jumping to ±50 at once. During the hold `trackFound` is `false` and `lineHeld` is `true`. A line that leaves at the frame **edge** is still pinned to ±50 at once.	Only if your code reacts to `trackFound == false` on the very first frame
Line lock follows the predicted position	The lock gate is centred on where the line is expected (last position + recent motion) and can no longer grow to the whole frame. The lock correction (`trackRelockFrames`) needs 3× as many frames to leave a valid-looking line.	No

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Calibration learns more	Besides the colours and the straight-ahead column, calibration now records the line **width**, how line-like its pixels are, and the **heading zero** (phone yaw on its mount). **Calibrate on a straight section, robot straight on the line.**	No
New `FrameData` fields	`steerOffsetTerm`, `steerHeadingTerm`, `headingDeg`, `headingConfidence`, `lineHeld`, `rejectedBands` — see *FrameData Fields*.	No (optional for a debug HUD)

Do I need to change my integration code? No. No public method, constant or field was removed or renamed — v2 only adds Config and FrameData fields, so existing code compiles and runs unchanged. What changes is the **behaviour**: steering now reacts to the robot's heading, distractors are ignored, and a line hidden mid-image is held briefly. To get the exact v1 behaviour back, set:

```
LineFollowerPipeline.Config v1 = new LineFollowerPipeline.Config();
v1.rejectDistractors  = false;   // original band selection (also turns off the hold)
v1.steerHeadingWeight = 0f;      // steering = offset only, exactly as in v1
pipeline.configure(v1);
```

Earlier changes

- **11 Oct 2026** — **Steering output is** `-50 … +50` (was `-100 … +100`): only the output is halved, the pipeline itself is unchanged. If your control program multiplies steering by a gain, **double the gain** to keep the same steering authority. `trackingMinValue` **default is** `5` (was `2`), so Tracking is `5 … 100`.
- **8 Oct 2026** — Automatic colour learning at calibration (`autoColor = true` by default; `trackIsBright` is now only the manual fallback); YUV 4:2:0 and ARGB input; Tracking ramps down with the remaining visible road; lock correction after turns (`trackRelockFrames`); `minContrast`; richer `FrameData` (calibration phase / progress / problem, learned colours, threshold, line width, frame size).

What This Library Does

`LineFollowerPipeline.java` is a pure-Java, zero-dependency line-following vision processor. You feed it raw camera frames — **grayscale, YUV 4:2:0 or ARGB** — and it returns two integer control values your robot can act on directly: a **steering direction** and a **speed/tracking signal**. No OpenCV, no Android APIs, and no native libraries are required.

Requirements

- Java 8 or higher
- One file only: `LineFollowerPipeline.java`
- No external dependencies

Minimal Integration (4 Steps)

1. **Create one instance (keep it alive for the session)**

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();
```

2 Calibrate — feed frames until it returns `true`

```
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    grayscaleBytes = camera.getGrayscaleFrame();
}
```

3 Call `processFrame()` on every camera frame

```
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

4 Read the two output values

```
int steering = result[0];  // use for direction control
int tracking = result[1];  // use for speed control
```

Have colour frames? Use `calibrateYuv420(y, u, v, w, h)` / `processFrameYuv420(…)` (Android camera) or `calibrateArgb(…)` / `processFrameArgb(…)` instead — colour input separates even line/floor colours of the same brightness. See *Input Formats*.

Calibration is required. `processFrame()` must not be called until `calibrate()` has returned `true`. The robot must be correctly placed on the line — straight, on a straight section, with floor visible on both sides — while calibrating. See *Calibration* below.

Input Formats

Each format has a matching calibrate / process pair. Use the **same format** for both.

FORMAT	CALIBRATE	PROCESS	DATA
Grayscale	`calibrate(gray, w, h)`	`processFrame(gray, w, h)`	`byte[] gray` — `w × h` luma bytes
YUV 4:2:0 *(recommended on Android)*	`calibrateYuv420(y, u, v, w, h)`	`processFrameYuv420(y, u, v, w, h)`	`y` — `w × h` bytes; `u`, `v` — `((w+1)/2) × ((h+1)/2)` bytes each
ARGB	`calibrateArgb(argb, w, h)`	`processFrameArgb(argb, w, h)`	`int[] argb` — `w × h` packed `0xAARRGGBB`

Important: all arrays must be packed with **no row padding** (and the U/V planes with no pixel interleaving). Android CameraX planes are padded / interleaved — strip that before passing the data in (see `copyLumaPlane` / `copyChromaPlane` in RoboPhoneApp's `MainActivity`). Arrays that are too short throw `IllegalArgumentException`.

Which format should I use?

- **Colour (YUV / ARGB)** separates any two colours that look different — even two colours of the **same brightness** (e.g. red on green).
- **Grayscale** works for any two colours that differ in **brightness** (white on black, black on blue, white on brown, black on white). Calibration reports `ISSUE_LOW_CONTRAST` and keeps retrying if the two colours have the same brightness.

Output Format

`processFrame()` returns a **reused** `int[2]` array. Copy the values immediately if you need to keep them — the same array is overwritten on the next call.

INDEX	NAME	RANGE	MEANING
`result[0]`	**Steering**	-50 … +50	`0` = robot is on the line and parallel to it. **Negative = turn left** (the line is to the left, or heads to the left). **Positive = turn right** (the line is to the right, or heads to the right). It combines where the line crosses the scan strip (offset) and which way the line runs ahead (heading) — see *Steering from the Whole* ⊥. `±50` toward the side where the line was lost (see *When the Line Is Lost or Hidden*).
`result[1]`	**Tracking**	5 … 100	`100` = straight path ahead, full speed. Drops as a turn or curve is detected, and ramps down linearly as the visible road ahead gets shorter (dead end, sharp corner). Never falls below `Config.trackingMinValue` (default **5**), so the robot is never sent a full stop. Set that field to `0` for the full 0–100 range.

Typical robot wiring:

```
robot.setSteeringAngle(result[0]);   // direction
robot.setSpeed(result[1]);           // speed — naturally slows before turns
```

Optional Configuration

To tune the pipeline for your camera mount, track, or robot body size, create a `Config` object and pass it **before the first frame**:

```
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
// --- apply your settings here ---
pipeline.configure(cfg);  // must be called BEFORE the first processFrame()
```

Calling `configure()` later discards the calibration (colours given with `setTrackColors()` are kept), so calibrate again afterwards. The **Range** column gives the values the code accepts or that make sense; values outside it are clamped or have no useful effect.

Line Colour

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`autoColor`	boolean	true	true / false	**Normal setup.** Learn the line and floor colours during calibration — any two different colours work and `trackIsBright` is ignored. `false` = classic grayscale mode using `trackIsBright`.
`trackIsBright`	boolean	true	true / false	**Manual fallback only** — not needed with `autoColor = true`. Used when `autoColor = false`, and before the colours have been learned (e.g. if `processFrame()` is called before calibration has finished). `true` = bright line on dark floor. `false` = dark line on bright floor.
`colorCalibFrames`	int	5	≥ 1	Frames sampled to learn the two colours.
`colorMinSeparation`	float	4.0f	≥ 0	Minimum line-vs-floor colour separation (in standard deviations of the colour noise). Below this, calibration discards the samples and retries.
`colorMinDistance`	float	12f	≥ 0	Minimum distance between the two learned colours (YUV grey-levels). Rejects "two colours" that are only sensor noise.
`colorMinThreshold`	int	112	0 … 255 (useful 64 … 192)	In colour mode the floor maps to 64 and the line to 192 internally; the threshold never drops below this so noisy floor pixels are never taken for the line.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`thresholdMargin`	int	0	grey-levels, ± (threshold kept in 15 … 240)	Bias added to the auto threshold. Increase if false positives occur. Decrease if the line is being missed.
`minContrast`	int	24	0 … 255	If the two halves of the Otsu split differ by less than this many grey-levels, the strip is a plain surface with **no line** (prevents "finding" a line in sensor noise). `0` disables.
`bottomIgnoreFrac`	float	0.15f	0 … 0.9	Fraction of the frame bottom to ignore — use this to exclude the robot's own body from the image. Increase until the chassis disappears from the scan. (RoboPhoneApp uses `0.25`.)

Scan Strip (H-ROI — Near Line / Steering)

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`hRoiCenterYFrac`	float	0.85f	0 … 1	Vertical position of the near scan strip as a fraction of frame height. `0.85` = 85% down from the top (near the bottom). The strip is moved up if it would overlap the ignored robot-body band.
`hRoiThickPx`	int	24	1 … frame height	Height of the scan strip in pixels.

Look-Ahead Column (V-ROI — Turn Preview / Tracking)

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`vRoiHalfWidthPx` CHANGED	int	80	≥ 1 px	Half-width in pixels of the look-ahead column. Increase on tight curves so the line stays inside the scan window. With `rejectDistractors` the window is automatically widened on rows where the line itself (plus a small margin) would not fit — e.g. wide tape at 960 × 1280.
`vRoiHeightFrac`	float	0.5f	0 … 1	How far ahead to look, as a fraction of frame height. `0.5` = half a frame ahead. Larger = more warning of far turns, but noisier.
`vRoiAdaptiveThreshold`	boolean	true	true / false	Gives the look-ahead its own Otsu threshold. **Recommended** — the far line is dimmer than the near line and benefits from separate calibration.

Camera Mount

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`cameraTiltDeg` CHANGED	float	45f	0 … 90 (degrees)	How far the phone's camera points below horizontal. Used for the perspective taper of the look-ahead, the look-ahead zero reference, the road-length slowdown, the line-width test of the look-ahead and the **heading** used by the steering (the line is projected onto the floor). Measure it on the real robot — a wrong value scales the heading part of the steering. Keep it large enough that the whole look-ahead lies below the horizon (floor only).
`verticalFovDeg`	float	60f	1 … 179 (degrees; phones ~55–65)	Vertical field of view of the camera (along the frame height). Most phone back cameras are ~55–65°. Used together with `cameraTiltDeg` for the floor projection.

Track Memory

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`trackMemory` CHANGED	boolean	true	true / false	Lock onto the line and only search a gate around its predicted position (last position + recent motion), so reflections elsewhere are ignored. `false` = search the whole width every frame.
`trackMemoryGateMult` CHANGED	float	3.0f	> 0	Gate half-width as a multiple of the measured line width (with `rejectDistractors`, never more than an acceptable band width, so the gate cannot grow to the whole frame). The gate is never wider than half the frame.
`trackMemoryMinGatePx`	int	48	≥ 0 px	Minimum gate width in pixels regardless of line width.
`trackLostGraceFrames`	int	8	≥ 0 frames	Frames to wait after losing the line before searching the full frame for a new one. At 30 fps, `8` ≈ 0.25 seconds.
`trackRelockFrames` CHANGED	int	6	≥ 0 frames	Lock correction. If a band that looks much more like the line (near the calibrated straight-ahead column, with the calibrated line width) is seen outside the lock for this many frames in a row, the lock moves to it — so a reflection or neighbouring tape grabbed after a turn cannot keep steering offset. With `rejectDistractors`, a locked line that still looks valid is only abandoned after **3×** this many frames. `0` disables.

When searching for the line (start-up, or after it was lost), the pipeline no longer simply takes the biggest band in view: it prefers the band nearest the calibrated straight-ahead column whose width matches the line width measured during calibration. While the locked line is lost at the frame edge, steering reads ±50 toward the side it left; a line that vanishes mid-image is held first (see *When the Line Is Lost or Hidden*).

Distractor Rejection (chair, bag, shadow, table edge) NEW

Objects with a colour close to the line's (a black chair leg on a black-tape track, a shadow on wood, the dark floor beyond a table edge) pass the threshold exactly like the line. With `rejectDistractors` on, a band only counts as the line if it **looks like the line learned at calibration**:

- **Width** — the band must be between `bandMinWidthMult` × and `bandMaxWidthMult` × the calibrated line width (the maximum is corrected for the line's slant in the image). Wide blobs, dark regions at the frame edge and thin specks are rejected.
- **Line-ness** — among acceptable bands, the one whose pixels are as line-like as the calibrated line wins (a shadow on wood only just passes the threshold; the tape is well past it).
- **Continuity** — while locked, the band nearest the *predicted* position wins (on the scale of one line width), and a band that is the line **merged** with a blob or shadow is cut back to the part that continues the line (its strong core, or the edge that sits where the line's edge was).
- **Look-ahead** — the row-by-row tracker of the V-ROI applies the same width test on every row (perspective-scaled), so it cannot step onto a nearby object; a run cut by the scan window is only used through the edge that continues the line.
- **Hidden line** — if the locked line disappears in the middle of the image (glare, a shadow or an object over it) the last steering and tracking are held for `trackHoldFrames`; a line that leaves at the frame edge (a real turn) is still pinned to that edge at once.
- **Relock** — the lock correction (`trackRelockFrames`) still exists, but a valid-looking locked line is only abandoned after three times as many frames, so a chair leg near the centre cannot steal the lock during a turn.

What you must ensure — the rules compare everything with the line measured at calibration, so:

- **Calibrate with the robot on the line**, straight, on a straight section, with floor visible on both sides, and **no distractor inside the scan strip** during calibration.
- **Use one tape width for the whole track** (a reasonable width: the line should cover well under half of the scan strip). Width changes of more than about 2× (`bandMaxWidthMult`) or less than 0.3× (`bandMinWidthMult`) are treated as "not the line". **Recalibrate** when you change tape.
- Set `cameraTiltDeg` / `verticalFovDeg` close to the real camera — the look-ahead width test uses them.
- Remaining limit: a dark object of **exactly the tape's width**, lying where the line is predicted to be, can still be taken for the line.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`rejectDistractors` NEW	boolean	true	true / false	Turn the rules above on. `false` = the original (v1) band selection (this also turns off the hold and the predicted gate).
`bandMaxWidthMult` NEW	float	2.0f	≥ 1.3 (useful 1.5 … 3)	Widest band accepted as the line, as a multiple of the expected (calibrated, slant-corrected) width. A real line stays within about ±30 %; a line merged with a blob is ≥ 2.5×.
`bandMinWidthMult` NEW	float	0.3f	0 … 1 (0 = no minimum)	Narrowest band accepted (fraction of the calibrated width); bands cut by the frame edge are exempt.
`trackHoldFrames` NEW	int	6	0 … trackLostGraceFrames	Frames the last output is held when the line vanishes mid-image (6 ≈ 0.1 s at 60 Hz). `0` = report the loss at once (v1). During the hold `trackFound` is `false` and `FrameData.lineHeld` is `true`. Only active with `rejectDistractors = true`.

Steering from the Whole ⊥ (offset + heading) NEW

The pipeline looks at the line as a ⊥ (an inverted "T"): the **base** is the horizontal scan strip (H-ROI), the **vertical part** is the line followed up the look-ahead (V-ROI). The v1 steering used only the base of the ⊥ — where the line crosses the scan strip — so a robot at 30° to the line and a robot straight on it read the **same** value if the line crossed the strip at the same point. The steering now also uses the line's **heading** from the vertical part of the ⊥:

```
offset   = (centroid − calibrated column) / (W/2) × 100                    (v1 steering)
heading  = 100 × L × tan(ψ − ψ₀) / (floor half-width seen by the H-ROI)    (ψ − ψ₀ limited to ±75°)
L        = steerLookAhead × (floor distance from the camera to the H-ROI)
steering (internal) = clamp(steerOffsetWeight × offset + steerHeadingWeight × c × heading, −100, +100)
result[0] = round(steering / 2)                                            → [-50 … +50]
```

- **W** is the frame width in pixels; the **centroid** is where the line crosses the scan strip; the **calibrated column** is the straight-ahead column measured during calibration (`FrameData.steerRefXFrac`).
- **ψ** is the direction of the line on the floor where it crosses the scan strip: the H-ROI centroid and the look-ahead rows on the line are projected onto the floor with the camera geometry (`cameraTiltDeg` , `verticalFovDeg` , exact pinhole model) and fitted with a parabola; its tangent at the strip is ψ. Positive = the line heads to the right.
- **ψ_0** is the heading measured during calibration (phone yaw on its mount), so a robot correctly placed on a straight line still reads `0` . Calibrate on a straight section; a calibration heading larger than `steerHeadingCalibMaxDeg` is ignored ($\psi_0 = 0$).
- Together the two parts are the lateral position of a **pure-pursuit target point** a distance L further along the line, in the same units as the v1 steering.
- **c** (the heading confidence, `FrameData.headingConfidence` , 0 … 1) fades the heading out when too little of the line is visible ahead (end of line, line leaving the frame) or when the line is not a smooth curve (a 90° corner, a junction) — the steering then falls back to the v1 offset only.

- **Signs**: both parts are positive when the robot must turn **right** (line to the right / line heading right). `FrameData.steerOffsetTerm` and `steerHeadingTerm` are the two weighted parts in output units (already halved), so before clamping and rounding `steering ≈ steerOffsetTerm + steerHeadingTerm`.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`steerOffsetWeight` NEW	float	1.0f	≥ 0 (keep 1)	Weight of the offset (v1) part.
`steerHeadingWeight` NEW	float	1.0f	≥ 0	Weight of the heading part. `0` **= exactly the v1 steering.**
`steerLookAhead` NEW	float	0.5f	> 0 (tested 0.25 … 1.0)	L as a multiple of the camera-to-strip floor distance. Larger = the heading counts more: a robot at an angle to the line straightens out sooner, but it runs a little more on the inside of tight curves and, with a high steering gain plus delay, can start to weave. `0.5` was the best balance in the closed-loop simulation. With the defaults, a robot at ±15° whose line crosses the strip at the calibrated column reads about ±8, at ±30° about ±17.
`steerHeadingMinRows` NEW	int	24	≥ 1 rows	Look-ahead rows on the line needed for the heading at full weight.
`steerHeadingCalibMaxDeg` NEW	float	10f	≥ 0 (degrees)	Largest heading accepted as the calibration zero.

Sensitivity & Tracking

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`minWeightFrac`	float	0.01f	0 … 1	Minimum fraction of scan strip pixels that must be "track" to trust the detection. Raise to reject weak signals.
`trackOffsetFullFrac`	float	1.0f	> 0 (usually 0.5 … 1)	How far the look-ahead line must lean (as a fraction of half-frame width) to push Tracking to its minimum. Lower (e.g. `0.6`) makes turns read earlier.
`trackLengthSlowdown`	boolean	true	true / false	Also limit Tracking by how much road is still visible ahead (floor distance, perspective-corrected). Approaching a dead end or a sharp corner, Tracking ramps down **linearly with the remaining road** instead of dropping from 100 to the minimum in one frame. A fully visible straight road still reads 100.
`trackLengthFullFrac`	float	0.8f	0 … 1	Fraction of the look-ahead distance the line must reach for the road-length part to read 100. Keeps the dim far edge from lowering Tracking on straight roads.
`trackingMinValue`	int	5	0 … 100	Floor for the Tracking output. `result[1]` will never go below this value, so the robot never receives a full stop signal. Set to `0` to allow the full 0–100 range.
`curveFullScaleDeg`	float	30f	—	**Not used** by the current code (no effect). Kept so existing code that sets it still compiles.

Tracking Minimum Value

By default Tracking **never reaches zero**: `result[1]` is clamped to a floor of 5, so the robot always receives at least a small speed signal. This prevents mechanical problems caused by a complete stop command.

VALUE	EFFECT
5 *(default)*	Tracking minimum is 5 — the robot always gets at least a small speed signal.

VALUE	EFFECT
10	Raise the floor further.
0	Disable the floor — allow the full 0–100 range.

```
cfg.trackingMinValue = 5;   // set before calling pipeline.configure(cfg)
```

Calibration Frames

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`calibFrames` CHANGED	int	10	≥ 1	Number of solid detections `calibrate()` collects before locking the straight-ahead zero reference (it takes their median), the line width, the line-ness level and the heading zero. Assumes the robot is correctly placed on the line. Increase for more robust calibration.

Calibration

Before calling `processFrame()` you must calibrate. Place the robot correctly on the line, then call `calibrate()` (or the YUV / ARGB version) once per camera frame until it returns `true`:

```
// Phase 1 — Calibration (at startup, or when the user presses calibrate)
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    // keep feeding frames until calibration is complete
    grayscaleBytes = camera.getGrayscaleFrame();
}

// Phase 2 — Normal operation
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

Calibration has two phases:

1. **Learn the colours** (`colorCalibFrames`, default 5 frames). The scan strip is sampled and split into two colour clusters (k-means). The **floor** is the cluster that fills both the left and right frame edges; the other is the **line**. If that fails — typically a shadow or uneven light making part of the floor almost as dark as a dark line — 3 and then 4 clusters are tried: the line is the cluster that forms one band and never reaches the frame edges, and all other clusters (lit floor, shadowed floor…) count as floor. A Fisher linear discriminant then gives the colour direction that best separates line from floor, and every later frame is converted to a "line-ness" image (line bright, floor dark) before the usual Otsu + line-lock processing. Skipped when `autoColor = false` or when the colours were given with `setTrackColors()`.
2. **Find straight-ahead** CHANGED Collects `calibFrames` (default 10) solid line detections — preferring the band where colour learning saw the line and, with `rejectDistractors`, of the width colour learning measured — and takes their **median** as: the steering zero reference (straight-ahead column); the typical **line width** and **line-ness level** (used by distractor rejection); and the **heading zero** ψ_0 (only when `steerHeadingWeight ≠ 0`): the median line heading of the frames that saw enough look-ahead. It is used only if at least half of the frames saw it and it is within `steerHeadingCalibMaxDeg`; otherwise $\psi_0 = 0$.

With the defaults calibration needs at least **15 frames** (5 + 10). It returns `true` once both phases are done. Progress and problems are reported in the callback (`calibPhase`, `calibProgress`, `calibIssue`):

CALIBISSUE	MEANING	WHAT TO DO
`ISSUE_NONE`	All good	—
`ISSUE_LOW_CONTRAST`	The two colours in view are too similar, or there is only one surface	Make sure the line is in the STEER strip; for same-brightness colours use YUV/ARGB input; even out the lighting
`ISSUE_NO_FLOOR`	No colour fills both frame edges	Place the robot **on** the line, with floor visible on both sides
`ISSUE_NO_LINE`	Colours learned, but no line band found this frame	Check the line is inside the STEER strip

Before you calibrate — checklist NEW

- The robot stands **on the line, straight**, on a **straight section** (the heading zero is learned here).
- The line is inside the scan strip, near the middle, with **floor visible on both sides** (both frame edges must show floor).
- No chair leg, bag, shadow edge or second tape inside the scan strip.
- The tape width is the one used on the whole track.

If calibration cannot complete, `calibrate()` simply keeps returning `false` — there is no timeout. Show `calibIssue` to the user, fix the cause (see the table) and keep feeding frames; call `requestRecalibration()` to start again from phase 1. Do **not** drive on `processFrame()` output before calibration is done: until then the steering zero is the image centre, there is no calibrated line width (so distractor rejection cannot work), and if the colours were not learned yet the frame is read as plain brightness with `trackIsBright` polarity.

Giving the colours yourself (optional)

```
pipeline.setTrackColors(0xFF000000 /* black line */, 0xFF1E3CC8 /* blue floor */);
pipeline.clearTrackColors();   // go back to learning them
```

`setTrackColors()` has no effect when `autoColor = false`, and throws `IllegalArgumentException` if the two colours are the same.

Re-calibrating mid-run

```
pipeline.requestRecalibration();
// then feed frames to calibrate() again before resuming processFrame()
```

Recalibrate whenever you move to a track with different colours, very different lighting or a different tape width.

When the Line Is Lost or Hidden NEW

SITUATION	STEERING RESULT[0]	TRACKING RESULT[1]	TRACKFOUND	LINEHELD
Line visible	offset + heading	from the look-ahead	`true`	`false`
Line disappears in the **middle** of the image (glare, shadow, object over it) — first `trackHoldFrames` frames (default 6)	**last value held**	**last value held**	`false`	`true`
Still hidden after the hold	±50 toward the side the line was last on	trackingMinValue (5)	`false`	`false`
Line leaves at the frame **edge** (sharp turn, robot drove off)	±50 toward that edge, at once	trackingMinValue (5)	`false`	`false`
Line comes back inside the lock gate (where it left) within `trackLostGraceFrames` (default 8)	normal again — the **same** line is re-locked	normal	`true`	`false`
Gone for more than `trackLostGraceFrames`	stays at the last edge value (±50, slightly less if the calibrated column is far off-centre)	trackingMinValue	`false`	`false`
A new line appears after that	full-width search: the band nearest the calibrated column with the calibrated width is taken	normal	`true`	`false`

Use `trackFound` (not `tracking == 5`) to show "TRACK LOST" — a hard turn also reads 5. With `trackMemory = false` there is no lock, no pinning and no hold: a missing line keeps the steering of its last position and tracking drops to the minimum. If calibration has not completed, see *If calibration cannot complete* above.

Runtime Utility Methods

```
int     fps       = pipeline.getFramesPerSecond();  // frames in the last completed second
long    frames    = pipeline.getTotalFrames();      // cumulative frame count since init
double  seconds   = pipeline.getElapsedSeconds();   // wall-clock seconds since first frame
boolean ready     = pipeline.isCalibrated();        // true once calibration is complete
boolean colours   = pipeline.isColorModelReady();   // true once colours are learned (or preset)
int     lineArgb  = pipeline.getLineColorArgb();    // learned line colour, 0xAARRGGBB
int     floorArgb = pipeline.getFloorColorArgb();   // learned floor colour, 0xAARRGGBB
boolean init      = pipeline.isInitialized();       // false before 1st frame / after release()
pipeline.requestRecalibration();  // re-learn colours + straight-ahead; then calibrate() again
pipeline.release();               // reset; the next frame re-initialises (calibrate again)
pipeline.init(960, 1280, cfg);    // optional: initialise now (size + config; drops calibration)
pipeline.setFrameCallback(cb);    // register overlay callback; pass null to unregister
```

Geometry getters (the same values the frame callback delivers — prefer the callback):

```
int     imgW    = pipeline.getImgW();                // frame size in pixels
int     imgH    = pipeline.getImgH();
int     stripTop = pipeline.getHRoiTop();            // scan-strip rows [top, bottom)
int     stripBot = pipeline.getHRoiBot();
int     vTop    = pipeline.getVRoiTop();             // far edge row of the look-ahead
int     vHalf   = pipeline.getVRoiHalfWidthPx();     // look-ahead half-width (config)
float   steerRef = pipeline.getSteerRefXFrac();      // calibrated straight-ahead columns
float   trackRef = pipeline.getTrackRefXFrac();
int     nPts    = pipeline.getLookAheadPointCount();  // dynamic road path (shared buffers)
float[] cols    = pipeline.getLookAheadColFracs();
float[] rows    = pipeline.getLookAheadRowFracs();
float   horizon = pipeline.getHorizonYFrac();        // floor horizon row from the mount tilt
float   scale   = pipeline.perspectiveWidthScale(0.5f); // relative apparent width at a row
```

Frame Callback — Overlay Geometry

Register a single callback once, and the library calls it automatically on **every calibration frame and every `processFrame()` frame**, passing a `FrameData` object with everything needed to draw a visual overlay and a status display: the H-ROI strip, the V-ROI look-ahead band, the dynamic road path, the live centroid, calibration progress and the learned colours.

No extra computation is added. The data already exists inside the pipeline — the callback just packages and delivers it (≈ 0.02 ms per frame). The callback runs on the thread that called calibrate / processFrame.

```
pipeline.setFrameCallback(data -> {
    // called automatically every frame
    // use data fields to draw your overlay
});

// To unregister:
pipeline.setFrameCallback(null);
```

Important: the same `FrameData` instance is reused on every call. Copy any fields you need to keep beyond the current frame.

FrameData Fields

FIELD	TYPE	DESCRIPTION
`imgW`, `imgH`	int	Frame size in pixels — the image all fractions refer to.
`hRoiTopFrac`	float	H-ROI top edge — fraction of frame height.
`hRoiBotFrac`	float	H-ROI bottom edge — fraction of frame height.
`vRoiTopFrac`	float	V-ROI far (top) edge — fraction of frame height.
`steerRefXFrac`	float	Calibrated straight-ahead column for steering — fraction of frame width.
`trackRefXFrac`	float	Calibrated straight-ahead column for look-ahead — fraction of frame width.
`halfBotFrac`	float	Near (bottom) half-width of the V-ROI box — fraction of frame width.
`halfTopFrac`	float	Far (top) half-width of the V-ROI box — fraction of frame width. Smaller than `halfBotFrac` due to perspective taper.
`ignoreTopFrac`	float	Top of the ignored robot-body band — fraction of frame height (`1` = nothing ignored).
`centroidXFrac`	float	Live detected line position — fraction of frame width (the real position, also during calibration). `-1` if the line was not found (also while it is held).
`trackWidthFrac`	float	Measured line width — fraction of frame width.
`tiltDeg`	float	Lean of the look-ahead line from vertical **in the image**, degrees (positive = leans right). For drawing only — the steering uses `headingDeg`. `0` before calibration or when the line is not found.
`lookAheadColFrac[]`	float[]	Column positions of the dynamic road poly-line, nearest to farthest. Array size **48**.
`lookAheadRowFrac[]`	float[]	Row positions of the dynamic road poly-line, nearest to farthest. Array size **48**.
`lookAheadPointCount`	int	Number of valid points in the path arrays. Only read up to this index.
`trackFound`	boolean	`true` when the line was detected in the H-ROI this frame. Use this for "TRACK LOST" — not `tracking > 5` (a hard turn also reads 5).
`isCalibrated`	boolean	`true` once calibration is complete.
`calibPhase`	int	`CALIB_LEARNING_COLORS`, `CALIB_FINDING_CENTER` or `CALIB_DONE`.
`calibProgress`	float	Overall calibration progress, `0 … 1`.
`calibIssue`	int	Current calibration problem (`ISSUE_*`, see *Calibration*).
`colorModelReady`	boolean	`true` when frames are processed in learned-colour mode.
`lineColorArgb`, `floorColorArgb`	int	Learned colours (`0xAARRGGBB`) — handy for UI swatches. `0` until learned.
`colorSeparation`	float	How well the two colours separate, in noise standard deviations.
`threshold`	int	Threshold used in the scan strip this frame; `-1` = no contrast (no line).
`steering` CHANGED	int	Steering value, `-50 … +50` — same as `result[0]`. `0` during calibration.
`tracking` CHANGED	int	Tracking value, `min … 100` — same as `result[1]`. `100` during calibration.
`steerOffsetTerm` NEW	float	Offset (v1) part of the steering, in steering output units (`steerOffsetWeight` applied). `0` during calibration and when the line is not found.

FIELD	TYPE	DESCRIPTION
`steerHeadingTerm` NEW	float	Heading part of the steering, in steering output units (already multiplied by `steerHeadingWeight` and `headingConfidence`). Before clamping and rounding, `steering ≈ steerOffsetTerm + steerHeadingTerm`. `0` during calibration and when the line is not found.
`headingDeg` NEW	float	Line heading relative to the robot on the floor, perspective-corrected, minus the calibrated zero. Degrees, positive = the line heads to the right. `0` when unknown (line not found, too little look-ahead, or `steerHeadingWeight = 0`).
`headingConfidence` NEW	float	Weight `0 … 1` the heading got this frame (`0` = too little / not a smooth line ahead → offset-only steering; also `0` when `steerHeadingWeight = 0`).
`lineHeld` NEW	boolean	`true` while the line is momentarily hidden mid-image and the last output is held (`trackHoldFrames`).
`rejectedBands` NEW	int	Number of candidate bands in the scan strip rejected or trimmed as distractors this frame (`0` with `rejectDistractors = false` and during calibration).

Drawing the overlay

Every field is a **fraction**, so convert to screen pixels first:

```
float screenX = data.centroidXFrac * screenWidth;   // any …XFrac field
float screenY = data.hRoiTopFrac   * screenHeight;  // any …YFrac / …TopFrac / …BotFrac field
```

Cropped previews: if the preview is scaled to fill the screen (Android `PreviewView` default FILL_CENTER), part of the image is cropped. Map with the same scale, or the overlay will drift: `scale = max(viewW / imgW, viewH / imgH)`, `x = (viewW − imgW·scale)/2 + xFrac·imgW·scale` (same for y). RoboPhoneApp's `OverlayView` does this.

H-ROI strip (a full-width horizontal band):

```
float top    = data.hRoiTopFrac * screenHeight;
float bottom = data.hRoiBotFrac * screenHeight;
canvas.drawRect(0, top, screenWidth, bottom, paint);
```

V-ROI trapezoid (perspective-tapered box — the far edge is narrower, and leans toward `trackRefX`):

```
float centre = data.steerRefXFrac * screenWidth;
float shear  = (data.trackRefXFrac - data.steerRefXFrac) * screenWidth;

// Near (wide) bottom corners — sit at the H-ROI top edge
float hTop = data.hRoiTopFrac * screenHeight;
float blX  = centre - data.halfBotFrac * screenWidth;
float brX  = centre + data.halfBotFrac * screenWidth;

// Far (narrow) top corners — shifted by shear so the box leans with the road
float vTop = data.vRoiTopFrac * screenHeight;
float tlX  = centre + shear - data.halfTopFrac * screenWidth;
float trX  = centre + shear + data.halfTopFrac * screenWidth;

// Connect the 4 corners:
//   bottom-left (blX, hTop) → bottom-right (brX, hTop)
//   → top-right (trX, vTop) → top-left (tlX, vTop) → close
```

Dynamic road band (bends through curves):

```
int n = data.lookAheadPointCount;
if (n >= 2) {
    for (int i = 0; i < n; i++) {
        float x = data.lookAheadColFrac[i] * screenWidth;
        float y = data.lookAheadRowFrac[i] * screenHeight;
        // connect points with a poly-line
        // i = 0 is nearest (bottom), i = n-1 is farthest (top)
    }
    // To draw as a band: trace the left edge (x - halfWidth) forward,
    // then the right edge (x + halfWidth) backward.
    // halfWidth tapers from halfBotFrac (near) to halfTopFrac (far).
}
```

Centroid indicator (live line position):

```
if (data.centroidXFrac >= 0) {
    float cx  = data.centroidXFrac * screenWidth;
    float top = data.vRoiTopFrac   * screenHeight;
    float bot = data.hRoiBotFrac   * screenHeight;
    // vertical line at cx, from top to bot
    // colour: green if data.trackFound, orange if not
}
```

State readout:

```
if (!data.isCalibrated) {
    status.setText("Calibrating " + Math.round(data.calibProgress * 100) + "%");
    // data.calibIssue explains a stall — see the Calibration table
} else if (data.lineHeld) {
    status.setText("Line hidden — holding S=" + data.steering);
} else if (!data.trackFound) {
    status.setText("Track lost");
} else {
    status.setText("Tracking  S=" + data.steering + "  T=" + data.tracking);
}
lineSwatch.setColor(data.lineColorArgb);   // learned colours
floorSwatch.setColor(data.floorColorArgb);
```

Steering debug readout NEW (new v2 fields — optional):

```
debug.setText(String.format("offset %.1f + heading %.1f (%.0f°, c=%.2f)  rejected=%d",
        data.steerOffsetTerm, data.steerHeadingTerm,
        data.headingDeg, data.headingConfidence, data.rejectedBands));
```

Full Example

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();
final int W = 960, H = 1280;    // frame size (RoboPhoneApp: CameraX 1280 × 960, rotated to portrait)

// Optional tuning
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
cfg.autoColor        = true;    // learn line + floor colours (any two colours)
cfg.cameraTiltDeg    = 45f;     // phone mount angle — measure it on your robot
cfg.bottomIgnoreFrac = 0.20f;   // hide robot body from scan
cfg.trackMemory      = true;    // lock onto line, ignore reflections
cfg.trackingMinValue = 5;       // never send a full stop
pipeline.configure(cfg);

// Register overlay callback — fires automatically on every frame
pipeline.setFrameCallback(data -> {
    myOverlay.drawHRoi(data.hRoiTopFrac, data.hRoiBotFrac);
    myOverlay.drawVRoi(data.vRoiTopFrac, data.steerRefXFrac,
                       data.trackRefXFrac, data.halfBotFrac, data.halfTopFrac);
    myOverlay.drawRoadPath(data.lookAheadColFrac,
                           data.lookAheadRowFrac,
                           data.lookAheadPointCount);
    myOverlay.drawCentroid(data.centroidXFrac, data.trackFound);

    if (data.isCalibrated) {    // drive only once calibration is complete
        robot.setSteeringAngle(data.steering);
        robot.setSpeed(data.tracking);
    }
});

// Phase 1 — Calibration (place robot correctly on the line first: straight, on a straight section)
Yuv f = camera.getYuvFrame();   // packed y / u / v planes
while (!pipeline.calibrateYuv420(f.y, f.u, f.v, W, H)) {
    f = camera.getYuvFrame();
}

// Phase 2 — Normal operation
while (running) {
    f = camera.getYuvFrame();
    pipeline.processFrameYuv420(f.y, f.u, f.v, W, H);
    // all results and overlay data delivered automatically via the callback
}
```

Behaviour Reference

SITUATION	STEERING	TRACKING
Straight line, centred	~0	~100
Line drifting to one side	± small	high
Curve approaching (line leaning)	varies	dropping toward the floor
Approaching a dead end / sharp corner	~0	ramps 100 → 5 with the remaining road
Robot at an angle to a straight line (line crossing the strip at the calibrated column) NEW	≈ ±8 at 15°, ≈ ±17 at 30° *(heading part)*	high

SITUATION	STEERING	TRACKING
Dark object / shadow beside or touching the line NEW	unchanged *(object rejected or trimmed off)*	unchanged
Line hidden for a moment in the middle of the image NEW	last value held for trackHoldFrames	last value held
Hard turn	±50	5–30
Line completely lost	±50	5 *(the floor)*
Back on a straight after a turn	returns to 0	~100

Recommended Starting Values (lecturer's test program) NEW

For the Blockly test program — **motor steering = (0 − Steering) × c** and **motor power = round(x × (Power / 8) + 5)**, where *Steering* is `result[0]` and *Power* is the Tracking value `result[1]` — the closed-loop simulation (`_investigation/CHANGES_v2.md`, §7) recommends:

SETTING	START WITH	NOTES
Steering gain **c**	`1.0`	Not `0.5`: the steering output was halved to ±50 (11 Oct 2026) after the program was written, so `0.5` now gives only half the steering authority. `0.5` is a gentler alternative on tracks without sharp corners. Keep `c ≤ 1.0` when power follows Tracking (`x = 1`).
Power mode **x**	`0` first (constant power 5), then `1`	With `x = 1` the power is about 6 … 18 and drops before turns. If the robot weaves with `x = 1`, use a larger divisor (e.g. `Power / 12`) before lowering `c`.
Offset **+ 5**	`5`	If the robot does not move at power 5 (motor dead zone), raise it to 6–8 rather than raising `c`.
Library	defaults	`steerHeadingWeight = 1`, `steerLookAhead = 0.5`. Set `cameraTiltDeg` to the real mount angle. If the robot cuts tight curves, lower `steerLookAhead` to `0.25`; if it hesitates when crossing the line at an angle, raise it to `0.75 … 1.0`.

These values come from simulation, not from the real robot — verify them on the track (and keep `x = 0` if the measured Blockly loop rate is well below ~15 Hz).

Notes

- The pipeline is **not thread-safe**. Call `calibrate()` / `processFrame()` (and `requestRecalibration()`) from a single thread only — e.g. have a UI button set a flag that the camera thread checks.
- The returned `int[]` is reused on every call. Copy the values if you need them past the next frame.
- Calibration assumes the robot is correctly placed on the line, with floor visible on both sides. If it is not, call `requestRecalibration()` once it is repositioned, then calibrate again. Recalibrate whenever you move to a track with different colours, different lighting or a different tape width.
- A line that jumps sideways by more than the line-lock gate in a single frame (`trackMemoryGateMult` × line width, around the line's predicted position) is treated as leaving the line — this is what makes the lock ignore reflections.
- The heading part of the steering relies on `cameraTiltDeg` and `verticalFovDeg` matching the real camera. The Android app runs at 960 × 1280 (CameraX picks 1280 × 960, rotated to portrait).
- Very harsh scenes can still defeat colour learning: e.g. a dark floor in deep shadow, with strong texture, where the tape is only ~10 grey-levels darker than the shadowed floor. Even lighting helps.
- Performance: ≈ 0.9 ms per frame in YUV colour mode at 960 × 1280 on a desktop CPU (several times more on a phone, still far below the 16.7 ms of 60 Hz); all per-frame buffers are pre-allocated (no garbage per frame, in all three input formats).

RoboPhoneApp — Android Reference Integration

`RoboPhoneApp/` runs the library live on the phone camera (CameraX → packed YUV 4:2:0 → `calibrateYuv420` / `processFrameYuv420`). Everything on screen comes from the `FrameCallback`:

- **Overlay** — cyan STEER strip with the detected line, magenta look-ahead band that bends along the line, dashed calibrated straight-ahead line, red ignored robot-body band; mapped with the preview's FILL_CENTER crop so it lines up with the camera image.
- **HUD** — STEER bar (−50…+50), TRACK bar (min…100 %), status (calibration progress and problems / TRACKING / TRACK LOST), FPS, learned LINE and FLOOR colour swatches, threshold and colour separation. The app does not show the new v2 `FrameData` fields yet.
- **RECALIBRATE** button — re-learns the colours and the straight-ahead reference (place the robot on the line first).

The app builds the library **from source** through a Gradle composite build (`includeBuild("../LineFollowerLibrary")`), so library changes show up in the app on the next build — no jar copying. Library tests: `cd LineFollowerLibrary && gradlew test` (format / colour unit tests plus camera-simulator scenes: shadows, wood grain, turns, line loss, distractor tapes, dead ends).

Unit tests and v2: all **26 tests pass** with the v2 defaults. The steering tests check the offset part (19 for a line 60 px right) and that the heading part steers the same way; `lostLineReportsTrackLost` checks the hold of `trackHoldFrames` frames before tracking drops to 5. Two tests (`v1SteeringIsOffsetOnly`, `v1LostLineDropsTrackingAtOnce`) check that `steerHeadingWeight = 0` and `trackHoldFrames = 0` give the v1 behaviour. `gradlew jar` and the app build do not run the tests.